COMP0249 Robot Vision and Navigation
Coursework 1

Amir Gerges, Manuel de Castro Ribeiro Jardim, Hashim,
and Cristhian Daniel Poveda Bravo

University College London

February 23, 2026

Group 4

1 GPS-enabled localisation system

1.1 Factor graph

In a simplified GPS-enabled localisation system, we initialise the vehicle at a 2D position with some orientation, and all subsequent states proceed according to the process model, with process noise. GPS measurements record a noisy reading of the vehicle state and follow the GPS measurement model.

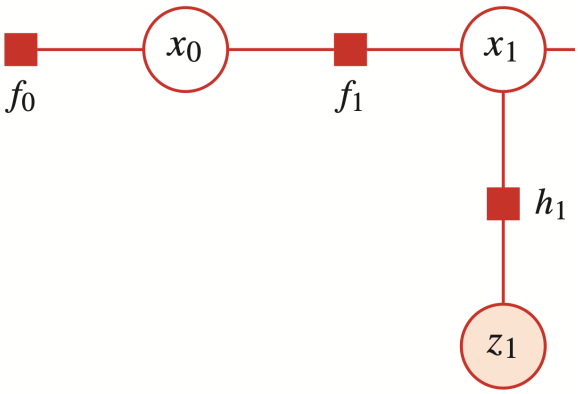


Figure 1: Factor Graph from Slides [1]

(a) Note that shaded circle z1 is not a vertex (it is a measurement) of the unary edge h1

The above factor graph consists of all the necessary vertices and edges required to represent this localisation system. We have three types of edges: unary initialisation edge f_0, binary process model edge f_k, and unary GPS measurement model edge h_k. We also have two types of vertices, namely the vehicle platform state x_k and the observed states z_k.

1.1.1 Initialisation: Edge $\mathbf{f}_0$ and Vertex $\mathbf{x}_0$

The initialisation of the `g2o` SLAM system is handled within the `G2OSLAMSystem.m` file, under the function `handleInitialisationEvent(obj, event)`. This function creates the first vehicle platform vertex $\mathbf{x}_0$ (or `vertexID` 1 using MATLAB convention). If the covariance of the initial state is zero, then the vertex is

fixed (i.e., its value will not be updated). However, if the covariance is non-zero, a unary prior condition edge is added instead; in our factor graph figure, this is represented by f_0, and in the MATLAB code, it is defined in `InitialPriorEdge.m`. This unary edge is implemented by setting the `obj.z` edge measurement to the prior belief of the initial state.

1.1.2 Vehicle State Process Model: Edges $\mathbf{f}_k$ and Vertices $\mathbf{x}_k$

Our SLAM system consists of a vehicle that moves in 2D - in the x direction and y direction - with orientation ϕ. The vehicle state at time step k is stored in the platform state vertices $\mathbf{x}_k = [x_k, y_k, \phi_k]^T$. During optimisation, the estimate is updated using an incremental update $\Delta\mathbf{x}_k = [\delta x, \delta y, \delta\phi]^T$ through the addition operator in the `oplus(obj, update)` function inside of the `PlatformStateVertex.m` file.

$$\mathbf{x}_k := \mathbf{x}_k + \Delta\mathbf{x}_k = \begin{bmatrix} x_k + \delta x \\ y_k + \delta y \\ \text{wrap}(\phi_k + \delta\phi) \end{bmatrix}.$$

Listing 1: PlatformStateVertex Addition

```
function oplus(obj, update)
    % OPLUS Apply an incremental update to the state estimate.
    %
    % Syntax:
    %   obj.oplus(update);
    %
    % Description:
    %   The incremental update to the platform state estimate.
    %   Because this is (x,y,theta), we have to normalize the
    %   heading afterwards.
    %
    % Inputs:
    %   update - (3x1 double)
    %       Small perturbed update to the state estimate. This will
    %       have the same state dimension as the vertex state.

    % Add the update
    obj.x = obj.x + update;

    % Wrap the angle to [-pi,pi]
    obj.x(3) = g2o.stuff.normalize_theta(obj.x(3));
```

Platform state vertices are connected within the `PlatformPredictionEdge.m` class. The process model is encoded by binary edges $f_1, f_2, \ldots, f_N$, which connect (i.e., constrain) the previous and subsequent vehicle platform states $\mathbf{x}_k - 1$ and $\mathbf{x}_k$. The edge measurement is the control input $\mathbf{u}$. The process model can be written as follows:

$$\mathbf{x}'_k = \mathbf{x}_k + \mathbf{M}(\phi_k)\left(\mathbf{u}_k + \mathbf{v}_k\right)$$

where

$$\mathbf{M}(\phi_k) = \Delta T \begin{bmatrix} \cos(\phi_k) & \sin(\phi_k) & 0 \\ -\sin(\phi_k) & \cos(\phi_k) & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$\mathbf{u}_k = \begin{bmatrix} s_k \\ 0 \\ \dot{\phi}_k \end{bmatrix}, \qquad \mathbf{v}_k = \begin{bmatrix} v_x \\ v_y \\ v_\phi \end{bmatrix}, \quad \text{where } \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k).$$

1.1.3 GPS Measurement Model: Edges $\mathbf{h}_k$

The GPS measurement model returns a measurement of the vehicle position in 2D, namely $[x_k, y_k]$, and is subject to Gaussian measurement noise.

$$\mathbf{z}_k^G = \begin{bmatrix} x_k \\ y_k \end{bmatrix} + \mathbf{w}_k, \qquad \text{where } \mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_G).$$

The GPS edge measurement `obj.z` are the observations $\mathbf{z}_k$, and are connected via unary edges $\mathbf{h}_k$ to the associated vehicle platform state $\mathbf{x}_k$, in MATLAB these unary edges are described in the `GPSMeasurementEdge.m` class. Because the GPS is simply observing the vehicle platform state (and the observations do not need to optimised), the measurements do not have separate vertices (hence the shading within the factor graph diagram)

1.2 Implementation

Note on dT

The g2o implementation we are given supports different delta T's for next platform state prediction, and it is passed as an argument for the `PlatformPredictionEdge` object constructor:

Listing 2: PlatformPredictionEdge Constructor

```
    methods(Access = public)
        function obj = PlatformPredictionEdge(dT)
            % PlatformPredictionEdge for PlatformPredictionEdge
            %
            % Syntax:
            %   obj = PlatformPredictionEdge(dT);
            %
            % Description:
            %   Creates an instance of the PlatformPredictionEdge object.
            %   This predicts the state from one timestep to the next. The
            %   length of the prediction interval is dT.
            %
            % Outputs:
            %   obj - (handle)
            %       An instance of a PlatformPredictionEdge

            assert(dT >= 0);
            obj = obj@g2o.core.BaseBinaryEdge(3);
            obj.dT = dT;
        end
```

Initial Estimate

The initial estimate function simply predicts the next state assuming no process noise, by applying the rotation matrix (calculated from the current heading angle) to the control input. Since `PlatformPredictionEdge` is a binary edge, it stores the previous state value in index 1, and the next state in index 2.

Listing 3: PlatformPredictionEdge initialEstimate

```
function initialEstimate(obj)
    % INITIALESTIMATE Compute the initial estimate of a platform.
    %
```

```
    % Syntax:
    %   obj.initialEstimate();
    %
    % Description:
    %   Compute the initial estimate of the platform x_(k+1) given
    %   an estimate of the platform at time x_(k) and the control
    %   input u_(k+1)
    x_k = obj.edgeVertices{1}.x;
    phi = x_k(3);
    M = obj.dT * [cos(phi) -sin(phi) 0; sin(phi) cos(phi) 0; 0 0 1];
    u = obj.measurement();
    x_pre_k_1 = x_k + M * u;
    obj.edgeVertices{2}.x = x_pre_k_1;
end
```

Compute Error

Computing the error in the prediction is equivalent to finding what the process noise was (if there were no process noise, the prediction from the process model would be exact). Starting from the process model, we rearrange the equation to explicitly solve for the process noise term $\mathbf{v}_k$.

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{M}(\phi_k)\left(\mathbf{u}_k + \mathbf{v}_k\right),$$

First, subtract the previous state:

$$\mathbf{x}_{k+1} - \mathbf{x}_k = \mathbf{M}(\phi_k)\left(\mathbf{u}_k + \mathbf{v}_k\right).$$

Multiplying both sides by the inverse motion matrix gives

$$\mathbf{M}(\phi_k)^{-1}(\mathbf{x}_{k+1} - \mathbf{x}_k) = \mathbf{u}_k + \mathbf{v}_k.$$

Rearranging yields an expression for the process noise:

$$\mathbf{v}_k = \mathbf{M}(\phi_k)^{-1}(\mathbf{x}_{k+1} - \mathbf{x}_k) - \mathbf{u}_k.$$

In the implementation, the inverse motion matrix $\mathbf{M}(\phi_k)^{-1}$ is denoted by $\mathbf{A}$, and we wrap the heading component of the state difference.

Listing 4: PlatformPredictionEdge computeError

```
function computeError(obj)
    % COMPUTEERROR Compute the error for the edge.
    %
    % Syntax:
    %   obj.computeError();
    %
    % Description:
    %   Compute the value of the error, which is the difference
    %   between the measurement and the parameter state in the
    %   vertex. Note the error enters in a nonlinear manner, so the
    %   equation has to be rearranged to make the error the subject
    %   of the formula.
    %
    % Using the formula from description: e(x,z) = inv(M) * (x_(k+1) - x_(k))
```

```
    x_k = obj.edgeVertices{1}.x;
    phi = x_k(3);
    A = (1/obj.dT) * [cos(phi) sin(phi) 0; -sin(phi) cos(phi) 0;0 0 1];
    x_k_1 = obj.edgeVertices{2}.x;
    u     = obj.measurement();

    error = A * (x_k_1 - x_k) - u;
    % do angle wrap correctly for rate residual:
    dphi = wrapToPi(x_k_1(3) - x_k(3));
    error(3) = (1/obj.dT) * dphi - u(3);
    obj.errorZ = error;
end
```

Calculating Jacobians

Since automatic differentiation is not provided within the supplied g2o coursework framework, we derived the Jacobians needed for optimisation analytically. These two Jacobians map the residual changes with respect to small perturbations in each connected state:

$$\mathbf{J}_{k-1} = \frac{\partial \mathbf{e}}{\partial \mathbf{x}_{k-1}}, \qquad \mathbf{J}_k = \frac{\partial \mathbf{e}}{\partial \mathbf{x}_k}.$$

Listing 5: PlatformPredictionEdge linearizeOplus

```
function linearizeOplus(obj)
    % LINEARIZEOPLUS Compute the Jacobians for the edge.
    %
    % Syntax:
    %   obj.computeError();
    %
    % Description:
    %   Compute the Jacobians for the edge. Since we have two
    %   vertices which contribute to the edge, the Jacobians with
    %   respect to both of them must be computed.
    %

    % warning('PlatformPredictionEdge.linearizeOplus: implement')
    x_k = obj.edgeVertices{1}.x;
    x_k_1 = obj.edgeVertices{2}.x;
    x1 = x_k(1);
    y1 = x_k(2);
    phi = x_k(3);
    x2 = x_k_1(1);
    y2 = x_k_1(2);
    phi2 = x_k_1(3);

    delta_x = x2 - x1;
    delta_y = y2 - y1;

    c = cos(phi);
    s = sin(phi);
    dT = obj.dT;
```

```
    obj.J{1} = (1/dT) * [-c, -s, -s*delta_x + c*delta_y;
                         s, -c, -c*delta_x - s*delta_y;
                         0, 0, -1];

    obj.J{2} = (1/dT) * [c, s, 0;
                        -s, c, 0;
                         0, 0, 1];

end
```

Diagrams Showing Consistency

Overall, the position and heading were within the range of the predicted covariances (within 2 sigma bounds) on most runs. On some runs, the position and heading were entirely within these bounds. However, on multiple rollouts, the position stays out of these bounds for most of the rollout. We noted that the consistency within the bounds was highly dependent on the random seed, which we controlled with the `rng(INT)` command at the beginning of the file.

Example Good Run:

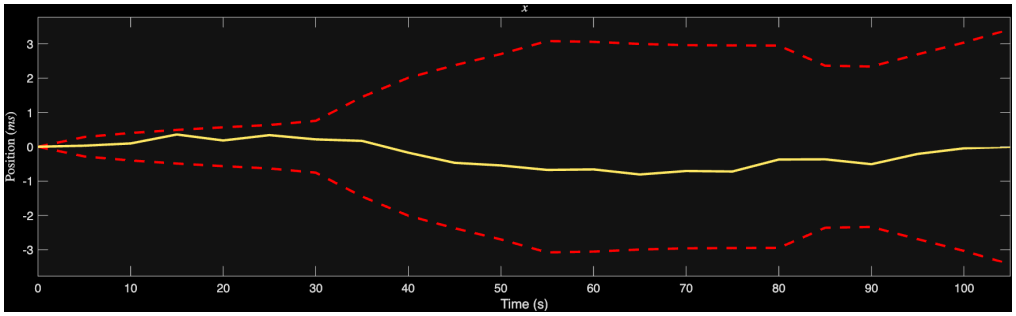


Figure 2: X Position

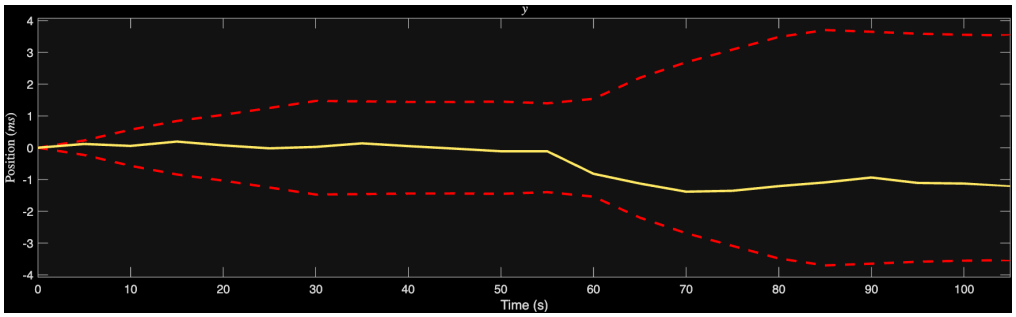


Figure 3: Y Position

Here x,y position was within 2 sigma bounds 100% of the rollout.

Example Bad Run:

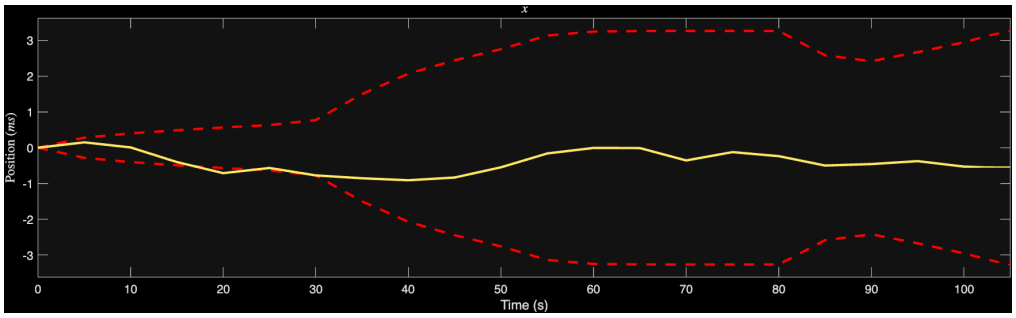


Figure 4: X Position

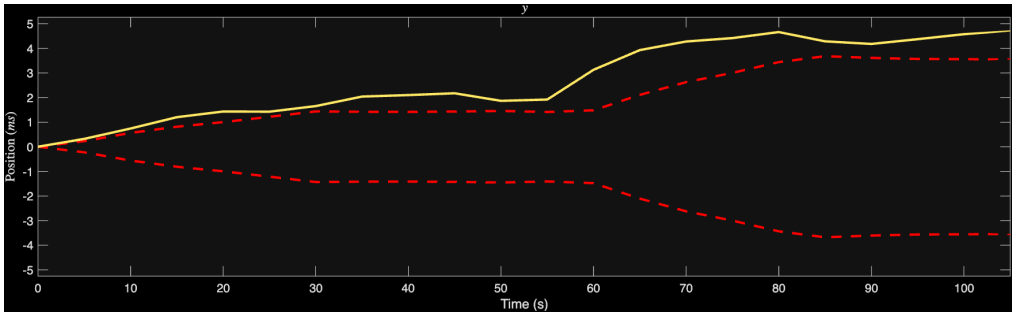


Figure 5: Y Position

We were unable to resolve these statistical issues. We were able to verify that the jacobians were being calculated correctly however, as replacing the `linearizeOplus` function with a finite difference implementation resulted in identical behaviour. Currently we are unsure what is causing the out of bounds behaviour. We speculate it could be due to a mistake in modeling the noise - e.g. the slip in y axis, perhaps a mismatch in the units of variance or a misunderstanding of covariance bounds. Without any clear mistakes in our implementation that we could find, and given that on most runs, the rollout positions were within the 2 sigma bounds 80-90% of the time, we proceeded with this implementation.

1.3 Graph optimisation

Chi-Squared Cost Function

The χ^2 value in the graph-based SLAM system is computed on a per-edge basis and represents a weighted measure of the residual error. Each edge contributes a cost based on the difference between the predicted and measured observation.

In the MATLAB implementation, this is defined in the `BaseEdge.m` class within the `chi2()` function:

Listing 6: Edge chi2 computation

```
chi2 = obj.errorZ' * (obj.Omega * obj.errorZ);
```

Mathematically, this corresponds to

$$\chi^2 = \mathbf{e}^T \Omega \mathbf{e} = \mathbf{e}^T \Sigma^{-1} \mathbf{e}.$$

Where $\mathbf{e}$ is the measurement residual and Ω is the information matrix (inverse of measurement covariance). Consequently, the χ^2 value increases when the measurement error increases or the measurement covariance decreases.

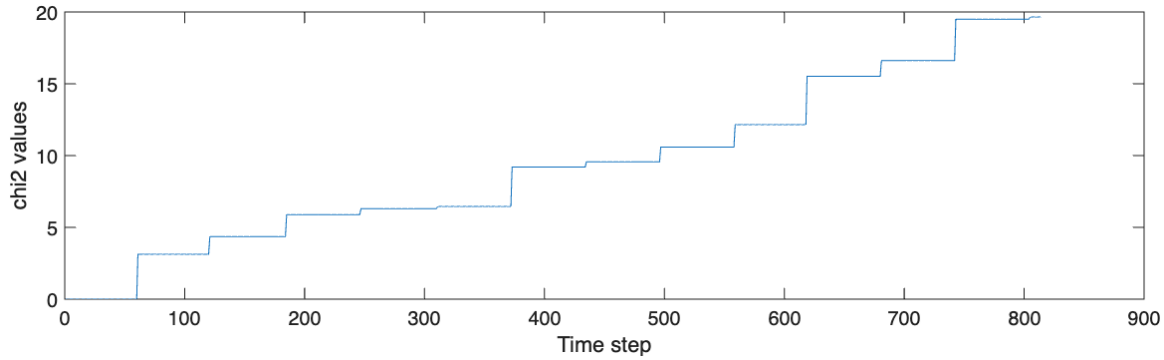


Figure 6: Chi 2

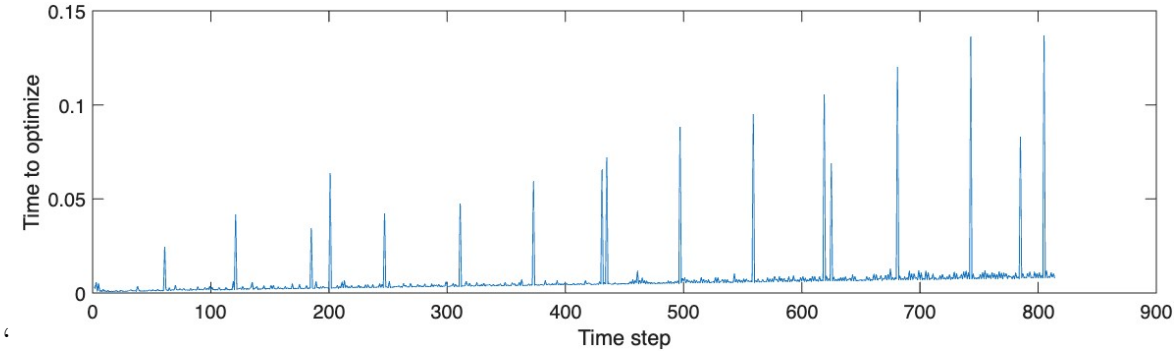


Figure 7: Solve Times

Observation

The χ^2 values start at zero, and only increase after 60 timesteps Then the χ^2 value stays constant as the process model predicts the next 60 steps, then jumps again, forming a staircase pattern of growth.

The solve times grow linearly as every vehicle platform state vertex is added. However, the solver time jumps up drastically every 60 time steps, and the solver time is far greater for that time step, before returning to the much more modest solve times. Amongst these spikes, each subsequent solver time spike also grows linearly.

Explanation

The χ^2 value starts at zero, and stays at zero because each state vertex already fulfils the process model constraint placed by the `PlatformPredictionEdge` binary edge. As such, the factor graph is already optimal and so the optimisation step run time is near zero. Once a GPS measurement arrives (every 60 time steps), a new measurement z_k arrives, along with a measurement model constraint h_k that is not satisfied automatically, and requires updating not only updating the state x_k, but all previous states from x_0 to x_k. Hence, the χ^2 spikes, and as does the solver time.

The optimisation step is done by solving the the linearised least squares equation [1]

$$F(x) = \sum_i e_i(x)^T \Omega_i e_i(x),$$

using the *Levenberg-Marquadt* algorithm, implemented in the solver as: `dX = (H + obj.mu * speye( n))\ b;`, which solves the system:

$$(H + \mu I)\Delta x = b,$$

where

- H is the Hessian (information) matrix - $H = J^T \Omega J$,
- Δx is the state correction applied at each iteration,
- μ is a damping parameter,
- $b = J^T \Omega e$

If the hessian were dense, then solving the system would be $\mathcal{O}(n^3)$ and prohibitively expensive. However, in this simple localisation example, the vehicle platform states only depend on previous states, and we do not even keep track of any landmarks. As such, the Hessian is very sparse, which is taken advantage of in the `SparseOptimizer` using the Sparse Cholesky decomposition, making the optimisation run in approximately linear time with respect to the number of edges.

2 Factor Graph SLAM

2.1 Factor graph

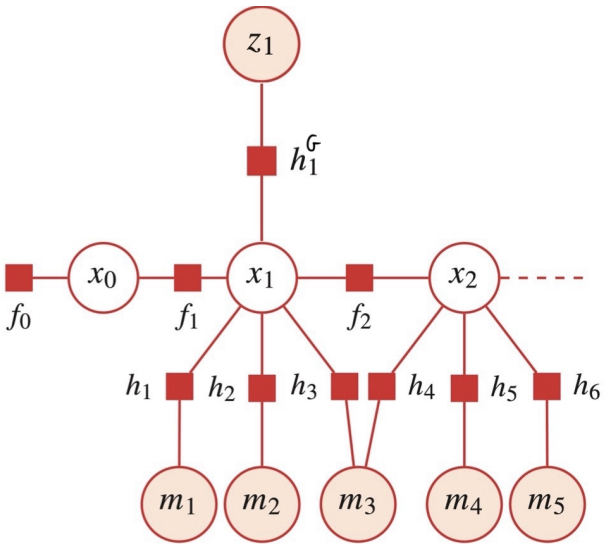


Figure 8: SLAM Factor Graph - Edited from [1]

We keep the initial prior edge f_0, platform state vertices $\mathbf{x}_k$, process model edges f_k, and GPS measurement edges $\mathbf{h}_k^G$ (where the superscript G denotes GPS measurements). To extend the graph to a SLAM formulation, landmark vertices $\mathbf{m}_i$ are introduced together with landmark observation edges $\mathbf{h}_i^L$.

Each landmark vertex $\mathbf{m}_i$ represents the 2D landmark position

$$\mathbf{m}_i = \begin{bmatrix} l_x \\ l_y \end{bmatrix}.$$

Since landmark states lie in Euclidean space, the addition operator is standard vector addition,

$$\mathbf{m}_i := \mathbf{m}_i + \begin{bmatrix} \delta l_x \\ \delta l_y \end{bmatrix}.$$

Unlike the platform state vertex, no angular component is present and therefore no angle wrapping is required. Consequently, the `LandmarkStateVertex` inherits the linear addition operator directly from the base class `g2o.core.BaseVertex`.

Landmark observations are modelled using the `LandmarkRangeBearingEdge`, which is a binary edge relating the vehicle platform state

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \phi_k \end{bmatrix}^T$$

(vertex slot 1) to the landmark position

$$\mathbf{m}_i = \begin{bmatrix} l_x & l_y \end{bmatrix}^T$$

(vertex slot 2) through the measurement

$$\mathbf{z}_k^L = \begin{bmatrix} r_k \\ \beta_k \end{bmatrix},$$

representing the range and bearing of the landmark expressed in the vehicle frame.

The measurement model follows simple geometric relationships:

$$r_k = \sqrt{(l_x - x_k)^2 + (l_y - y_k)^2},$$

$$\beta_k = \text{atan2}(l_y - y_k,\ l_x - x_k) - \phi_k.$$

The landmark measurements are assumed to be corrupted by zero-mean Gaussian noise,

$$\mathbf{z}_k^L = \begin{bmatrix} r_k \\ \beta_k \end{bmatrix} + \mathbf{w}_k^L, \qquad \mathbf{w}_k^L \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_k^L).$$

2.2 Implementation

To finish the factor graph SLAM system, we had to implement the `LandmarkRangeBearingEdge` class, which handles the observation model for the 2D range-bearing sensor. The following is a break down of how the three methods were implemented.

Initial Landmark Estimation

In the `initialEstimate` function, the goal is to estimate the landmakr position (lx, ly)

Listing 7: initialEstimate implementation

```
function initialEstimate(obj)

    pose = obj.edgeVertices{1}.x;   % [x; y; theta]
    z = obj.measurement();          % [range; bearing]

    r = z(1);
    beta = z(2);
    phi = beta + pose(3);

    lx = pose(1) + r * cos(phi);
    ly = pose(2) + r * sin(phi);

    obj.edgeVertices{2}.x = [lx; ly];
```

First, we acess the robot's current pose (which gives x, y, and θ) and the raw sensor measurement, which gives the range r and bearing β. To get the global heading to the landmark, ϕ, the robot's heading is added to the bearing, so $\phi = \theta + \beta$. After that, standard trigonometry was used to project the range and find the landmark's coordinates: $l_x = x + r\cos(\phi)$ and $l_y = y + r\sin(\phi)$. These values are then saved into the landmark's vertex.

Error Computation

For `computeError`, we calculate the difference between a predicted measurement (the range and bearing we expect to see given our prediction of the landmark position) and the actual measurement z_k.

Listing 8: copmuterError implementation

```
function computeError(obj)

    % Predict a measurment z_hat
    pose = obj.edgeVertices{1}.x;   % [x; y; theta
    landmark = obj.edgeVertices{2}.x; % [lx; ly]
    dx = landmark(1) - pose(1);
    dy = landmark(2) - pose(2);
    r_hat = sqrt(dx^2 + dy^2); % predicted range
    beta_hat = atan2(dy, dx) - pose(3); % predicted bearing

    % get the actual measurement z
    z = obj.measurement(); % [range; bearing]
    r_measured = z(1);
    beta_measured = z(2);
    e = [r_measured - r_hat; wrapToPi(beta_measured - beta_hat)];
    obj.errorZ = e;
```

To get the predicted measurement the Euclidean distance between the estimated robot pose and the landmark for the predicted range $\hat{r}$ were calculated. For the predicted bearing $\hat{\beta}$, the `atan2` function was used on the differences in y and x, and the robot's heading subtracted. The error vector e is simply the true measurement minus this prediction $(z - h(x))$. One crucial part here is wrapping the angular error using the `wrapToPi` function. If this isn't done, an angle difference like crossing the 180° to −180° boundary will look like a massive error, and the optimizer will fail trying to "unwind" a non-existent mistake.

Jacobian Derivation

The `linearizeOplus` method is where the Jacobians for the error with respect to the landmark measurement vertex m_i - $[lx, ly]$ and state vertex x_k - $[x, y, \theta]$ is calculated. We derived the jacobians analytically. Small checks (`if q < 1e-12`) were added as a safety measure. This prevents divide-by-zero error if the vehicle is extremely close to the landmark.

Listing 9: linearizeOplus implementation

```
function linearizeOplus(obj)

    pose = obj.edgeVertices{1}.x;   % [x; y; theta]
    x = pose(1); y = pose(2); theta = pose(3);
    landmark = obj.edgeVertices{2}.x; % [lx; ly]
    lx = landmark(1); ly = landmark(2);
    dx = lx - x; dy = ly - y;
    q = dx^2 + dy^2;
    if q < 1e-12
        q = 1e-12;
    end
    r = sqrt(q);

    Je_x = [dx/r, dy/r, 0;
            -dy/q, dx/q, 1];
```

```
    Je_m = [-dx/r, -dy/r;
            dy/q, -dx/q];

    obj.J{1} = Je_x;
    obj.J{2} = Je_m;
```

Proof of Correctness (Zero Noise SLAM)

With zero noise enabled, we expect the g2o Factor-Graph SLAM and the EKF-SLAM systems to perform identically.

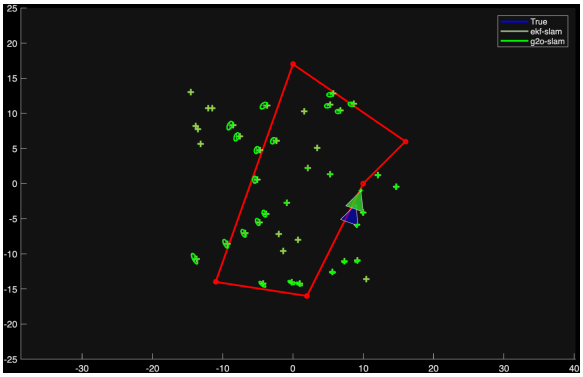


Figure 9: EKF-SLAM vs G2O Factor Graph SLAM - No Noise

We can see clearly, that the mean and covariance bounds for all landmarks for both the EKF and Factor-Graph SLAM systems are identical, lining up over each other.

2.2.1 Comparison to Q1b

Based on the generated plots for the system running with noise, the estimation errors for the position states (x and y) oscillate but predominantly remain contained within the predicted $\pm 2\sigma$ covariance bounds indicated by the red dashed lines. This observation indicates that the SLAM system is statistically consistent, meaning the optimizer's calculated uncertainty accurately reflects the true estimation error throughout the trajectory.

These results extend the findings from Q1b, where a pure GPS-enabled localization system was analyzed. In Q1b, the position and heading errors were also observed to be consistent with the predicted covariance values, remaining within the 2σ bounds on most runs. The trends observed in this scenario prove that despite transitioning from a simple localization problem to a complex SLAM problem with unknown landmark states and non-linear range-bearing measurements, the g^2o factor graph formulation successfully maintains its statistical consistency.

Furthermore, the χ^2 cost value increases monotonically over time in a step-like progression, reaching approximately 2200 by the end of the simulation. Alongside this, the time required to optimize the graph increases as the simulation progresses, exhibiting a distinct sawtooth pattern that peaks at roughly 3.5 seconds.

The spikes in solve time, and the jumps in χ^2 values occur when landmarks are observed, similar to the way that the GPS localisation χ^2 values jumped and solver time spiked when a GPS measurement is observed. The general upward trend, is caused by the increasing size of the factor graph; as the robot navigates, new poses and landmark observation edges are constantly added, which naturally increases both the total residual error (as the number of edge factors increases) and the computational load required to solve the sparse linear system (as the size of the Hessian increases).

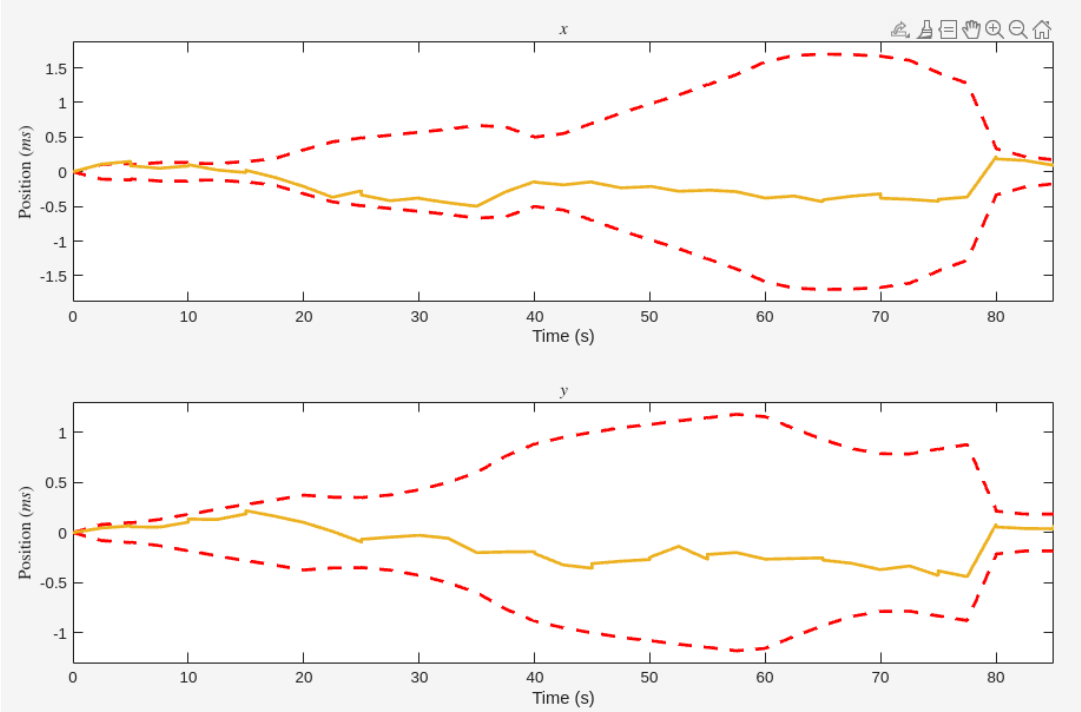


Figure 10: Chi2 and optimization during noisy run

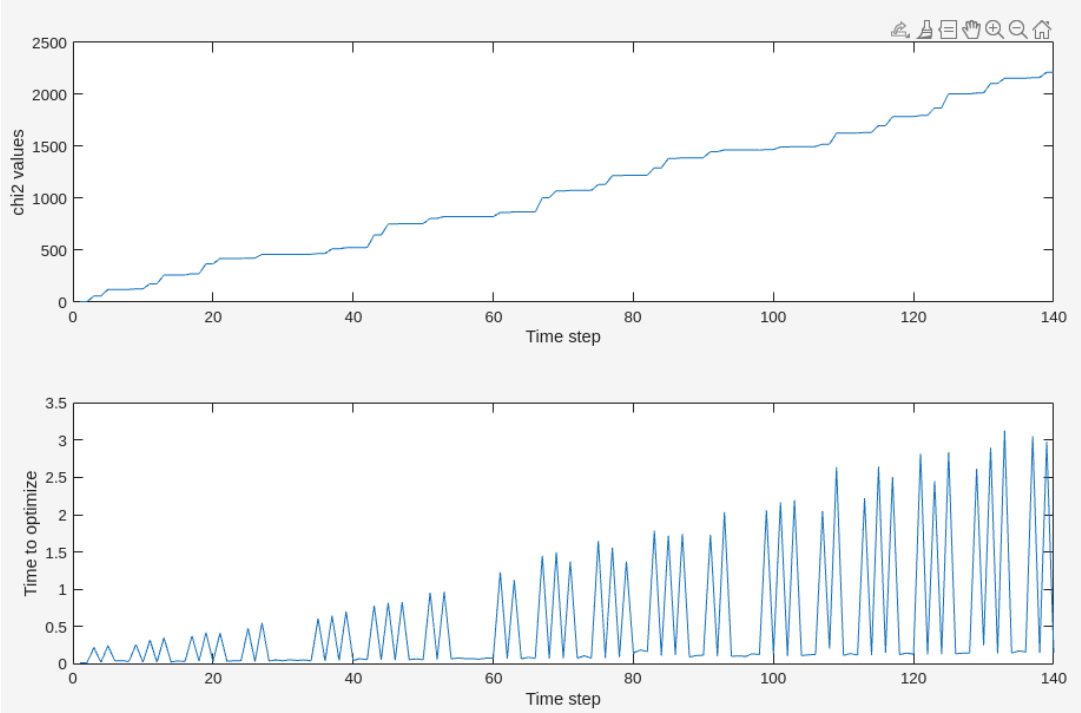


Figure 11: Error bounds during noisy run

2.3 Larger scale scenario

Evidence of Algorithm Consistency

In state estimation, an algorithm is considered consistent if its actual estimation error aligns with its calculated uncertainty bounds. The generated plot clearly illustrates that the EKF-SLAM system (light green) fails this requirement. For many of the landmarks along the edges of the map, the true landmark positions (represented by green crosses) fall completely outside of the EKF's estimated covariance ellipses. This indicates that the EKF has become overconfident; it calculates a very small uncertainty despite having

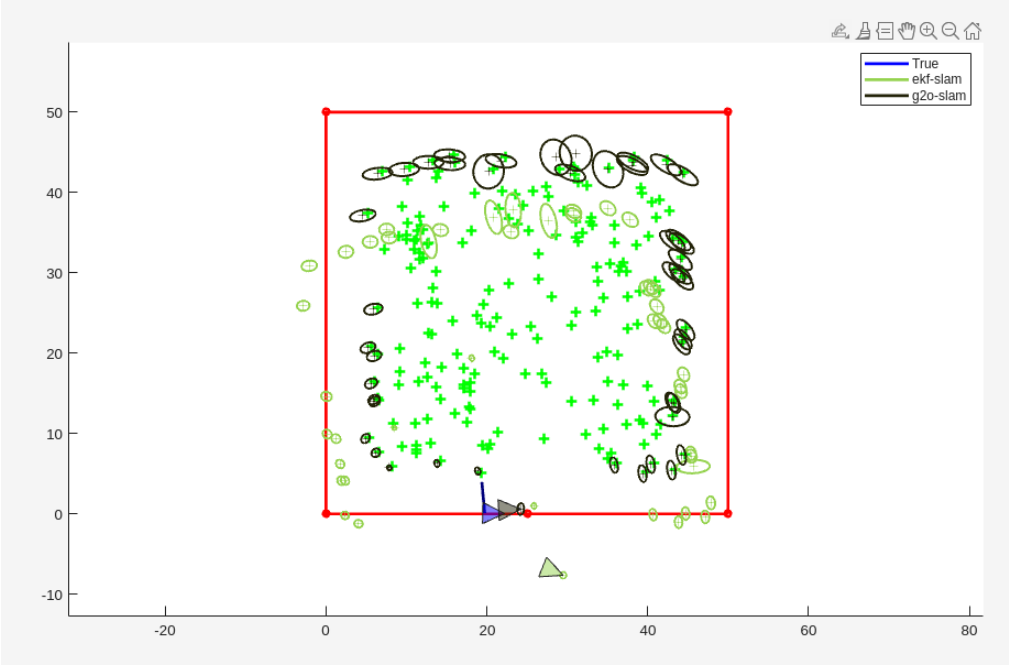


Figure 12: Trajectory and landmark estimates for EKF and g2o SLAM under restricted detection range.

a large actual error. Additionally, the EKF vehicle's estimated pose has drifted significantly away from the true blue path.

In contrast, the factor graph system (g^2o, shown in black) maintains consistency. Its covariance ellipses correctly encompass the true landmark positions, and the estimated vehicle trajectory remains aligned with the ground truth.

Impact of a Smaller Detection Range

A smaller sensor detection range means the robot observes fewer landmarks at any given time and tracks them for shorter durations before they leave the field of view. This scarcity of measurement data disproportionately degrades the performance of the EKF compared to the factor graph algorithm.

The EKF relies heavily on continuous measurement updates to keep odometry drift in check. With a restricted detection range, the robot experiences longer periods where it must rely purely on dead reckoning, allowing the state error to grow rapidly. Because the EKF is a recursive filter that linearizes around the current estimate only once, a bad estimate caused by prolonged drift locks in permanent linearization errors. When a landmark is finally observed again, the filter cannot correctly reconcile the new data with its heavily drifted state, leading to the inconsistency observed in the plot.

On the other hand, the g^2o factor graph approach retains the entire history of vehicle poses and measurements. Even when data is sparse, it performs batch optimization across the entire trajectory. If a sequence of measurements eventually provides enough information to correct the map, the graph optimization relinearizes past states and pulls the entire historical trajectory back into place. This ability to revise past estimates makes the graph-based system far more resilient to the temporary tracking losses caused by a short detection range.

Adjusting the Detection Range

Increasing the `detectionRange` parameter in the `scenario.json` file to a value of 14 successfully brings the estimates of both SLAM algorithms into alignment. As the updated plot demonstrates, the EKF system (light green) no longer diverges. Instead, its estimated trajectory and landmark covariance ellipses closely track both the ground truth (blue) and the factor graph estimation (black).

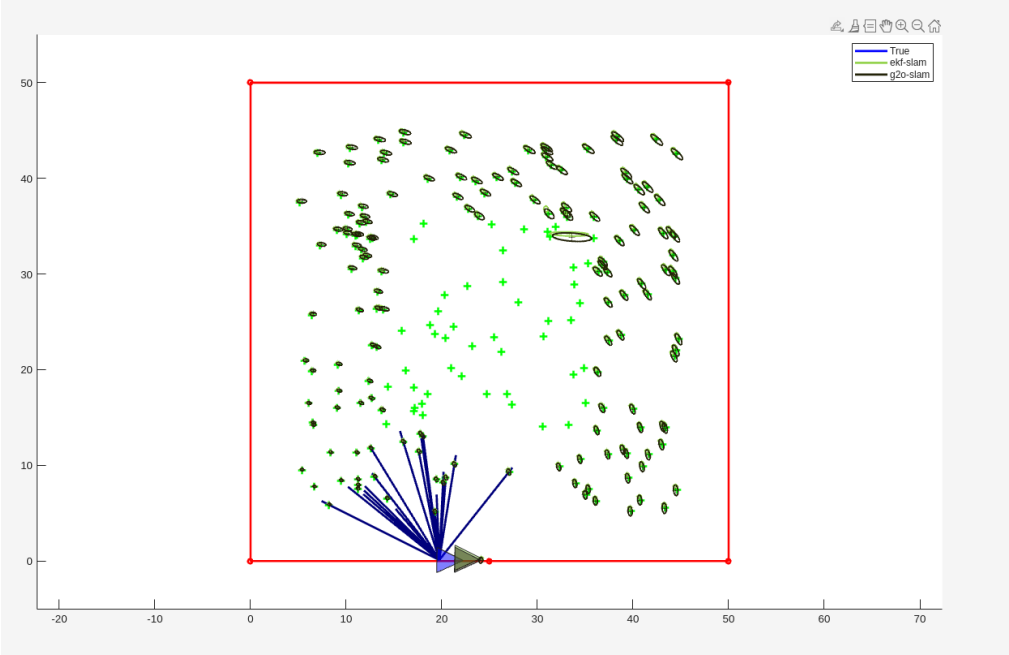


Figure 13: SLAM results after increasing the detection range to 14. Both EKF and factor graph estimates now align closely with the ground truth.

This improvement happens because a larger detection range allows the robot to observe more landmarks simultaneously and keep them in view for a longer duration. For the EKF, having a steady, dense stream of measurement data is crucial. It ensures the filter can constantly correct small odometry drifts before they have a chance to accumulate into large, unrecoverable linearization errors. Because the EKF does not have to rely on blind dead reckoning for long stretches when the range is extended, it stays consistent and produces results that are practically on par with the more robust factor graph approach.

3 Graph Pruning

This section will examine the performance of this pruning strategy, which ensures that each landmark has no more than four observation constraints within the factor graph. The performance of the full factor graph SLAM estimator (g2o-slam) will be compared to that of the pruned factor graph SLAM estimator (g2op-slam), with ekf-slam used as a baseline. The pruning strategy has been used to prevent the growth of measurement factors caused by repeated observations of landmarks, which can slow down non-linear optimisation.

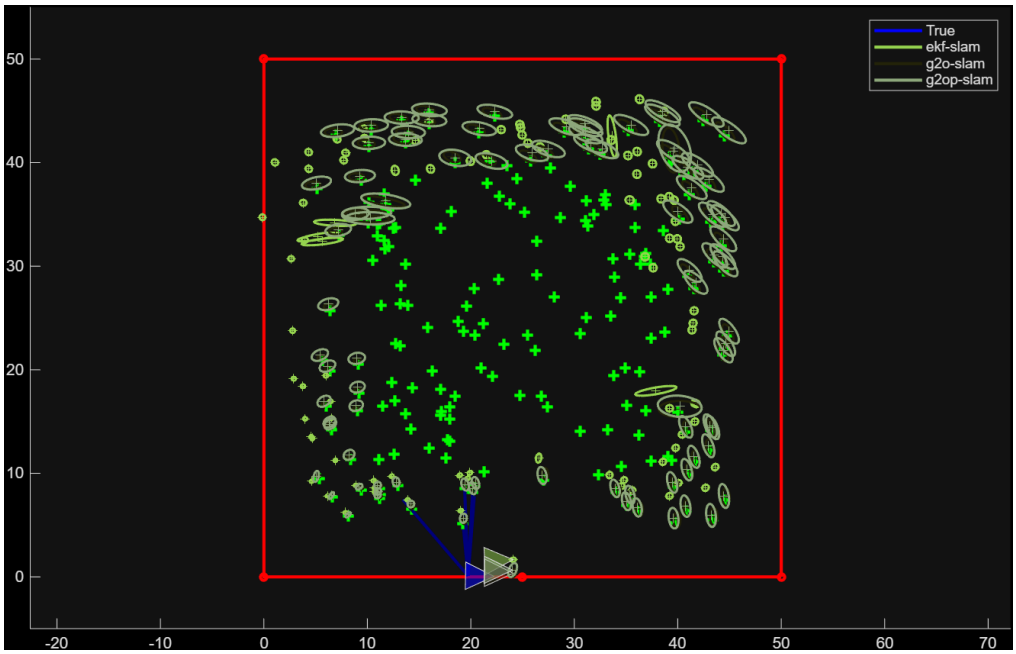
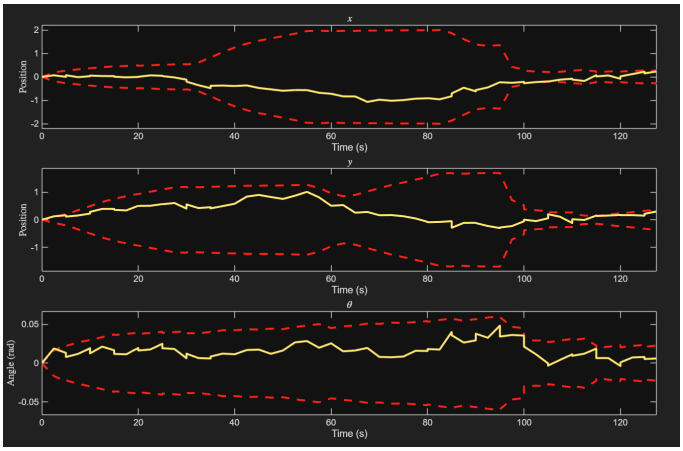


Figure 14: Estimated trajectory and landmark map for ekf-slam, g2o-slam (full), and g2op-slam (pruned).

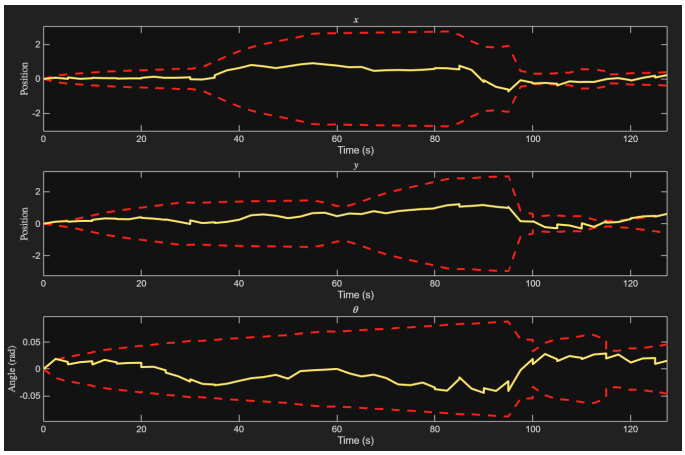
Figure 14 indicates that the overall trajectory and map obtained by the three estimators are similar. This indicates that the pruning strategy has not resulted in any obvious divergence. However, the main differences can be observed by comparing the estimation error with the uncertainty bounds. Additionally, the optimisation process can be compared.

3.1 Standard Deviation

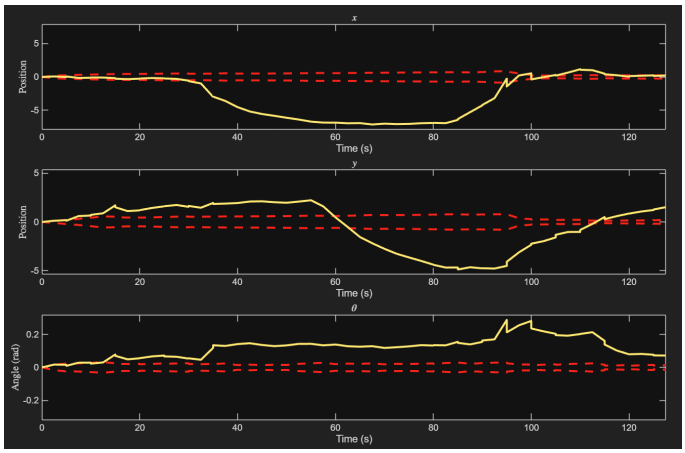
Figure 15(a) indicates that the full factor graph estimation (g2o slam) is generally consistent, with all errors contained within the $\pm 2\sigma$ region. This is evidence that the optimiser's covariance is well calibrated to the actual error, and this is further reinforced by the mean uncertainties: $2\sigma_x = 1.0119$, $2\sigma_y = 0.9235$, and $2\sigma_\theta = 0.0396$ rad. Although there is some discrepancy between the two axes, this is to be expected, as the configuration of the landmarks and the movement of the robot do not constrain these axes equally throughout the entire scenario. Furthermore, the heading is well calibrated, as bearing measurements are very sensitive to changes in orientation, and with so many landmarks detected, the graph is able to correct the θ state well.



(a) g2o-slam



(b) g2op-slam



(c) ekf-slam

Figure 15: State error and $\pm 2\sigma$ bounds for (a) g2o-slam, (b) g2op-slam, and (c) ekf-slam.

Figure 15(b) indicates that after pruning, the uncertainties in all states increase, but the estimator still behaves consistently. The mean bounds increase to $2\sigma_x = 1.3411$, $2\sigma_y = 1.2659$, and $2\sigma_\theta = 0.0571$ rad. This is because, after pruning, there is one less range-bearing measurement, and each of these is an additional constraint reduced between the pose and the landmark. In the full graph, seeing the same landmark multiple times is beneficial, as it helps to prevent drift, but limiting each to four measurements reduces this effect, making the estimation more uncertain between strong corrections. Again, the same reasoning is applied to the increase in $2\sigma_\theta$, as there is less information about the bearing.

Figure 15(c) indicates that, although the EKF estimation results in much lower uncertainties of $2\sigma_x = 0.4715$, $2\sigma_y = 0.4918$, and $2\sigma_\theta = 0.0205$ rad, the error is not contained within the $\pm 2\sigma$ region. This is evidence of overconfidence and inconsistency. Inconsistency is to be expected, as the EKF's covariance is often too low after multiple linearisations, especially when it is difficult to maintain correlations over long ranges, which is evidenced by the error plot.

3.2 Optimisation Time

Figure 16 indicates that the value of the chi^2 increases over time for both factor graph estimators. This is expected since additional residual terms are continually added to the cost function as new poses and measurements are included. The final value of the chi^2 was higher for the full graph (1575.19) than it was for the pruned graph (560.75). The main reason for this was that the full graph has many more measurement edges than the pruned graph, which contributed to the final value of the chi^2. This, however, does not necessarily mean that the full solution was worse.

The optimisation time plot shows how pruned graphs can improve performance. The mean optimisation time was reduced from 0.5878 s to 0.4992 s, which is about a 15% improvement. The reason that optimi-

sation time was reduced was that every optimisation step involves solving a linear system that consists of all constraints on the graph. The landmark observation edges introduce additional dependencies between the pose and landmark variables. If these landmarks are observed multiple times, this can increase the optimisation time since it increases the linear solve. Pruning removes many of these couplings, so optimisation becomes faster. The speed-up is not huge because the number of pose states and motion edges is unchanged, and those still grow over time.

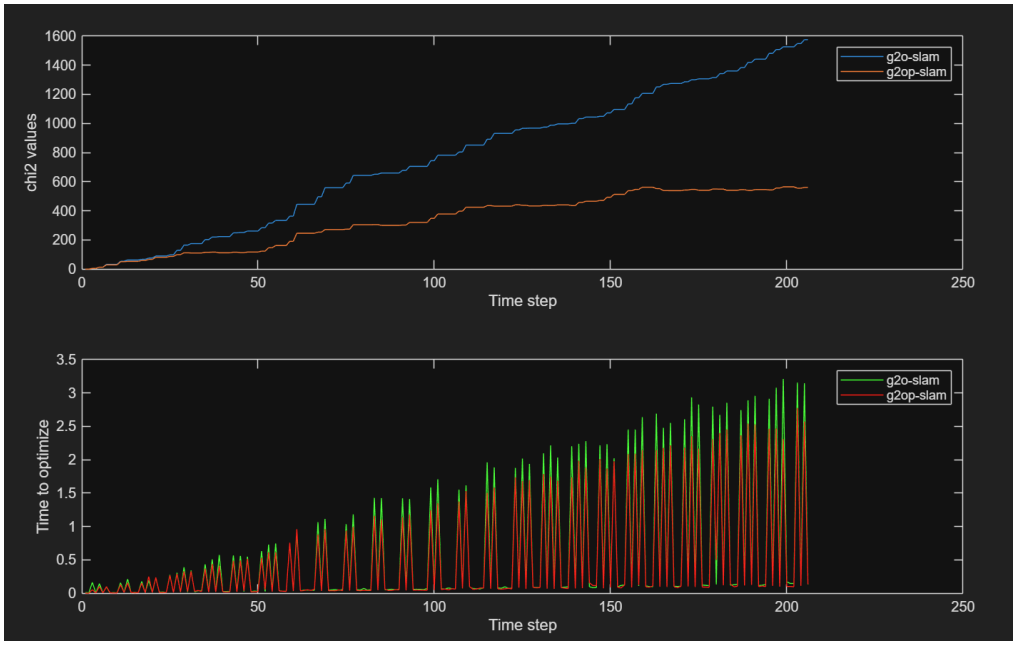


Figure 16: Chi^2 cost and optimisation time per step for g2o-slam (full) and g2op-slam (pruned).

3.3 Graph Size

Pruning only helps if it reduces the main source of graph growth without breaking the constraint structure needed for stable estimation. As can be observed in the graph counts, the pruning maintains the state size while removing many of the measurement constraints. Both estimators have 1295 platform state vertices and 1294 platform prediction edges, so the trajectory backbone is identical. Both also have 91 landmark vertices, meaning the number of landmark states is unchanged. The principal difference in the two estimators can be observed in the number of landmark range bearing observation edges, which are 796 in the full graph, while in the pruned graph, they are 336. Therefore, a total of 460 measurement edges are removed, which is about 58% of the landmark observation constraints.

This explains the earlier results. Removing observation edges reduces the amount of measurement information available to tighten the estimate, which increases marginal uncertainty. This is exactly what is seen in Fig. 15(b), including the increase in heading uncertainty from ($2\sigma_\theta = 0.0396$) rad (full) to ($2\sigma_\theta = 0.0571$) rad (pruned). At the same time, removing edges reduces computation because the landmark observation edges are the main source of extra coupling between the pose chain and the landmark map. When a landmark is observed several times, it becomes a high-degree node that connects many poses through one landmark. This increases the complexity of the sparse linear system. By pruning the edges, the system reduces the number of these connections and the fill-in. This results in a reduction of the average optimisation time, as seen in the reduction from 0.5878 s to 0.4992 s.

The speed-up is useful but not massive because pruning does not reduce the number of poses. Even after pruning, the system still estimates the full trajectory and keeps the motion constraints, so the problem continues to grow as time increases. Therefore, the pruning effect is most noticeable in an environment with many re-observations and loop closures (dense landmarks and many revisits), where the number of measurement edges would be high. In an environment with few repeated observations, the pruning effect would be less significant.

Table 1: Graph structure comparison between the full and pruned factor graphs.

Element Type	g2o-slam	g2op-slam
PlatformStateVertex	1295	1295
LandmarkStateVertex	91	91
PlatformPredictionEdge	1294	1294
LandmarkRangeBearingEdge	796	336

4 Fixing vertices

4.1 Chi2 Values and Optimisation Times

Figure 17 show the chi2 values and optimisation times for the baseline method and the optimised version using fixed vertices.

The chi2 values follow the same behaviour as the baseline SLAM system. They increase directly proportional to the factor graph size over time, as more observations are done, and nodes are created. After the optimised version starts fixing vertices, the chi2 values are not affected immediately compared to the baseline method because the difference between the observations and the created map remains the same. A difference is introduced during loop closure events around simulation step 150. When the robot revisits a landmark, the fixed states in the graph prevent a global alignment of the past estimations and the new measurements, causing the chi2 values to increase. Specifically, the fixed-vertices method considers the first observation of the landmark as ground truth. Therefore, the error is propagated to the new observation, and the discrepancy between the new observations and the map increases. As a result,t the chi2 values for the optimised approach are higher than the chi2 values of the non-optimised version due to the added error to the newly added observations in the local area. The difference in chi2 values continues while there are fixed vertices in the graph. However, the final chi2 value for both methods is the same for the last time step when all nodes are unfixed because both factor graphs become the same, thus all nodes are adjusted in the same way.

The optimisation time for both methods creates staggered lines over time, with peaks appearing when a new keyframe is identified. Overall, the optimisation time for both methods is the same before nodes are fixed. After time step 50, there is a clear difference because the Hessian matrices' size becomes different due to the number of active vertices on each factor graph. The optimisation time for the baseline method continues growing, whilst the optimisation time for the smaller graph gets to a stable plateau that continues over the whole execution. In the last time step, where all vertices are unfixed, there is a significant peak in the execution time for the optimised version, which is slightly higher than the last baseline calculation time. This is due to the optimised method having accumulated a bigger error in a larger number of nodes. As a result, when all the states are unfixed, the solver requires more time to match all the factors and nodes in a consistent way to produce the final result, whilst the baseline method performs a final gradual adjustment.

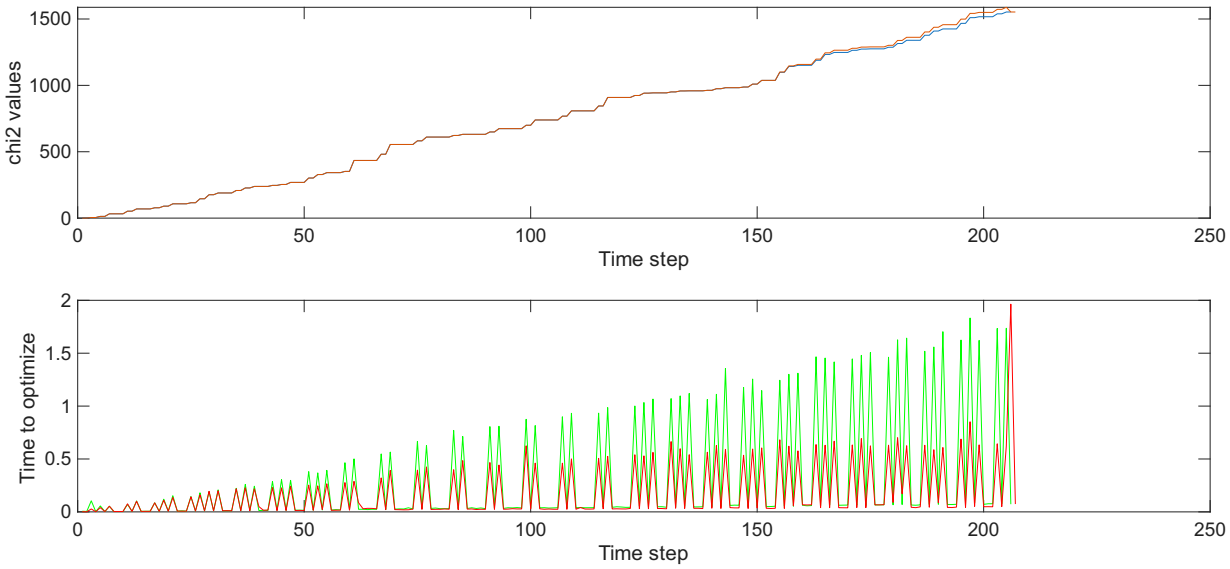


Figure 17: Chi2 values and optimisation times for fixed-vertices

4.2 Position and covariance estimates

Figures 18 and 19 display the position estimates error and uncertainty for the baseline g2o system and the fixed-vertices g2o. The estimated covariances gradually grow for the baseline method as the robot navigates in the environment. On the other hand, the optimised method assumes zero uncertainty for the fixed vertices, which artificially shrinks the total covariance of the factor graph. This makes the whole system overconfident in the estimated position of the landmarks. As a general behaviour, the optimised approach is more biased in the position estimations compared to the baseline. The non-optimised system is able to reduce the error in estimation by adjusting the whole graph when a new observation is included. This keeps all the nodes closer to the ground truth. Conversely, the optimised method cannot improve accumulated drift, and the estimated position for several landmarks remains the same since the first estimation. Yet, this method is more confident than the baseline, presenting smaller covariance areas. This inconsistent behaviour is confirmed when the ground truth of a landmark is outside of the uncertainty region of the estimation. The baseline method, most of the time, can get the true position of the landmarks within its uncertainty interval, whilst the optimised method considers a smaller uncertainty area and loses the true position of the landmark. It is harder to get new landmarks right because fixing vertices eliminates the correlation between old nodes and new ones. Therefore, it is not possible to update the whole map to get to the global best estimation of the environment.

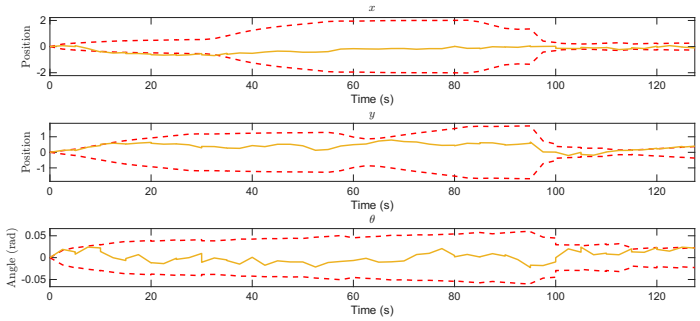


Figure 18: Position estimates and uncertainty bounds for the baseline g2o SLAM

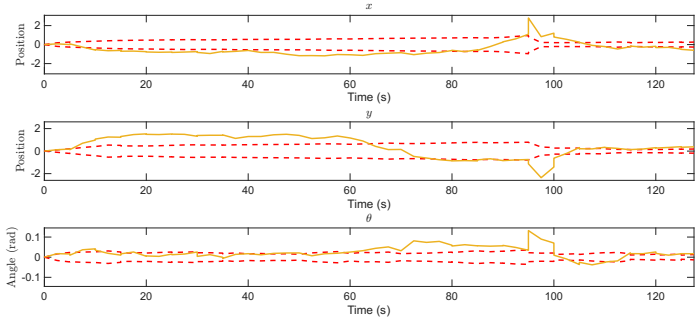


Figure 19: Position estimates and uncertainty bounds for the optimised g2o SLAM

4.3 Strategy success

The optimised version is partially successful in the SLAM task. On the one hand, it is able to accurately navigate and model the local area of the robot. However, the model fails in getting a precise global map of the environment. Due to the unaware biased landmarks, the system creates non-existing obstacles in the map and misses some actual ones in the final map. On the other hand, this optimisation strategy is necessary to achieve real-time execution because the optimisation time becomes stable and predictable, which otherwise would grow indefinitely. For instance, it is required to find the right balance between how many active vertices are left, in order to get the most precise map as a possible while ensuring real-time execution at a certain frequency. In summary, this approach enables a bounded optimisation time per time step, which guarantees that the system will not suffer from a computational crash in the long term, regardless of the environment size.

5 Appendix

Listing 10: G2O SLAM Factor Graph Initialisation

```
function success = handleInitializationEvent(obj, event)
    % HANDLEINITIALIZATIONEVENT Handle the initialization event.
    %
    % Syntax:
    %   slamSystem.handleInitializationEvent(event);
    %
    % Description:
    %   Handle the initialization event. data is assumed to be x0,
    %   and covariance P0. The initialized flag is also set to true
    %   to enable the estimator to run in full mode.
    %
    % Inputs:
    %   event - (ebe.core.Event)
    %       The intialization event.
    %
    % See Also:
    %   SIMULATOR

    % Create the first vertex, set its estimate to the initial
    % value and add it to the graph.
    obj.currentPlatformVertex = cw1.drivebot.graph.PlatformStateVertex(obj.currentTime);
    obj.currentPlatformVertex.setEstimate(event.data);
    obj.graph.addVertex(obj.currentPlatformVertex);

    % Set the book keeping for this initial vertex.
    obj.platformVertexId = 1;
    obj.platformVertices{obj.platformVertexId} = obj.currentPlatformVertex;

    % If the covariance is 0, the vertex is known perfectly and so
    % we set it as fixed. If the covariance is non-zero, add a
    % unary initial prior condition edge instead. This adds a soft
    % constraint on where the state can be.
    if (det(event.covariance) < 1e-6)
```

```
        obj.currentPlatformVertex.setFixed(true);
    else
        initialPriorEdge = drivebot.graph.InitialPriorEdge();
        initialPriorEdge.setMeasurement(event.data);
        initialPriorEdge.setInformation(inv(event.covariance));
        initialPriorEdge.setVertex(obj.currentPlatformVertex);
        obj.graph.addEdge(initialPriorEdge);
    end

    obj.initialized = true;

    success = true;
end
```

References

[1] S. Saeedi, "Comp0249 robot vision and navigation: Lecture 08 – factor graph slam." Lecture slides, 2026. Department of Computer Science, University College London.